



2024

FPGA Task Force
OPUS Lab

On-Chip Simulated Annealing

Abdelrahman S. Abdelrahman^{1,*}, Navid Anjum Aadit^{1,†}, and Kerem Y. Çamsari^{1,‡}

¹ *Department of Electrical and Computer Engineering,
University of California, Santa Barbara, Santa Barbara, CA, 93106, USA*

* abdelrahman@ucsb.edu

† maadit@ece.ucsb.edu

‡ camsari@ece.ucsb.edu

Contents

1	Introduction	1
2	System Architecture	1
2.1	Beta Counter Module	3
2.2	Beta Scaler Module	4
3	FPGA vs CPU Annealing ($L = 3$)	5
3.1	Linear Schedule	5
3.2	Geometric Schedule	6
4	FPGA vs CPU Annealing ($L = 5$)	6
4.1	Linear Schedule	6
4.2	Geometric Schedule	7
5	Benchmarking Analysis	7
6	IP Usability	8
7	Conclusion	9
8	Recommendations	9
A	Simulated Annealing Sampling Algorithms (CPU and FPGA)	10

List of Figures

1	A schematic of system architecture	1
2	Simulated Annealing on CPU and FPGA using 30 beta values that are linearly spaced. (Left) Excitation energy versus time steps. (Right) Success probability versus energy. . . .	5
3	Simulated Annealing on CPU and FPGA using 100 beta values that are linearly spaced. (Left) Excitation energy versus time steps. (Right) Success probability versus energy. . . .	5
4	Simulated Annealing on CPU and FPGA using 100 beta values that are geometrically spaced. (Left) Excitation energy versus time steps. (Right) Success probability versus energy.	6
5	Simulated Annealing on CPU and FPGA using 100 beta values that are linearly spaced. (Left) Excitation energy versus time steps. (Right) Success probability versus energy. . . .	6
6	Simulated Annealing on CPU and FPGA using 100 beta values that are geometrically spaced. (Left) Excitation energy versus time steps. (Right) Success probability versus energy.	7
7	Simulated annealing benchmarking using 25 β values and $L = 3$: Time/experiment averaged over 1000 independent experiments vs. the total # of sweeps/experiment using offline, CPU, and online annealing (the proposed design).	7

1 Introduction

This report presents a novel design for accelerated simulated annealing on Field Programmable Gate Array (FPGA) hardware chips, where the communication between the FPGA and MATLAB is strategically delayed until the end of the annealing process. The document offers a detailed overview of the proposed system's architecture and outlines its key algorithms. Furthermore, the dynamics of the system are verified by analyzing energy plots and success probability metrics, using two distinct instances of 3D-Spin Glass problems with different sizes ($L = 3$ and $L = 5$). The benchmarking analysis section provides an in-depth look at the improvements achieved by this design. For ease of use, the IP usability section includes a step-by-step guide on implementing the proposed IP for general problem settings. The report concludes by suggesting potential enhancements to the current design, laying the groundwork for future research and development in this field.

2 System Architecture

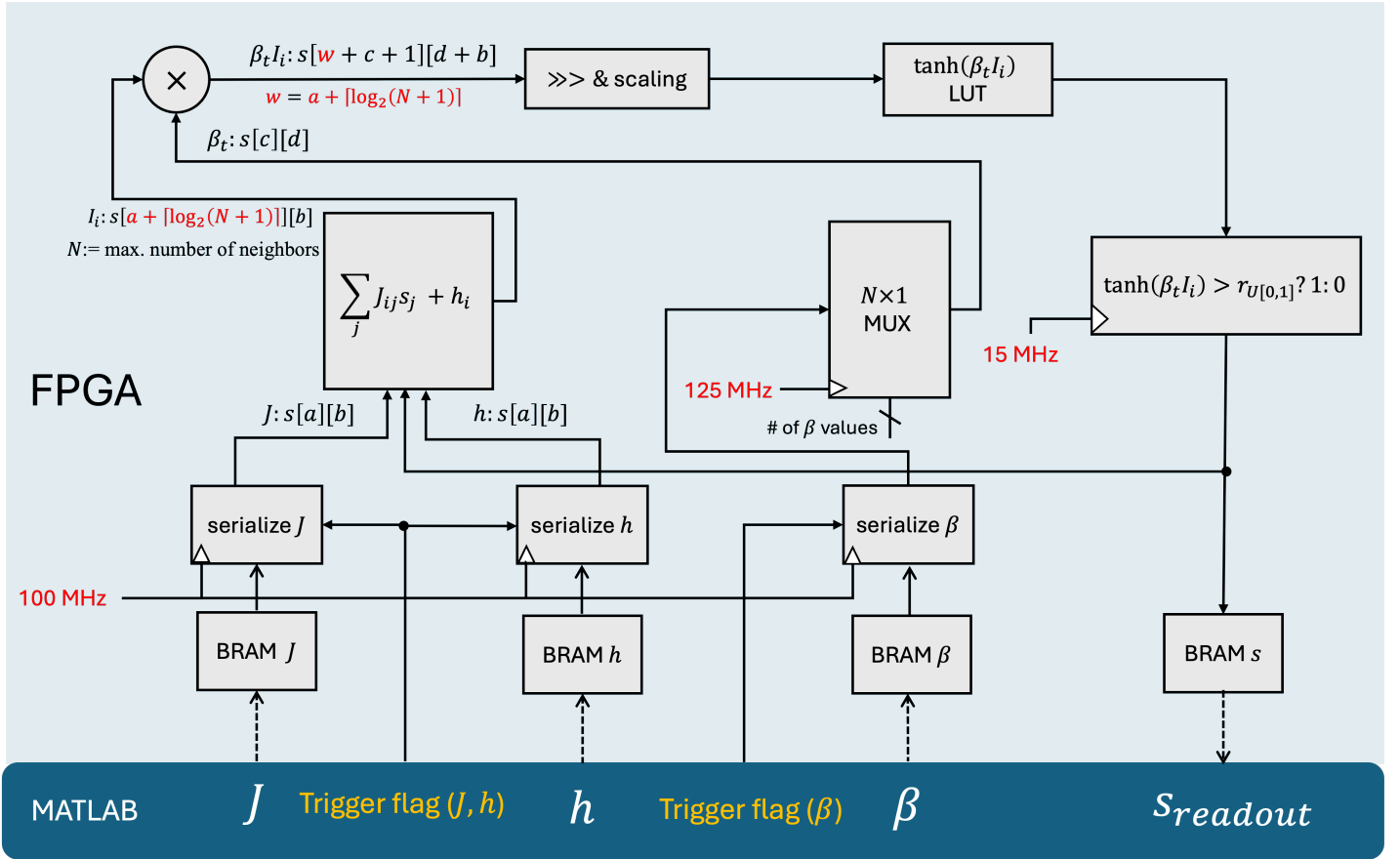


Figure 1: A schematic of system architecture

In Figure 1, the system's block diagram is presented, showcasing the data flow and adjustments in fixed-point precision. The process begins with the serial loading of problem parameters—namely, the adjacency matrix (J), bias vector (h), and annealing schedule (β)—from MATLAB to the FPGA. Additionally, two

flags are transmitted from MATLAB to indicate the readiness of the parameters for retrieval from the Block RAMs (BRAMs). The "serialize $\mathbf{X}$ " blocks play a crucial role in reading the parameters stored in BRAMs and reconstructing them in their original format for use in subsequent modules. For instance, the serialize J block reconstructs the J matrix, while the serialize h and serialize β blocks reconstruct the vectors h and β , respectively. These blocks are activated by shifting the stored data in the BRAMs only when their corresponding trigger flags are set to high.

Given that J and h define the problem and remain constant throughout the annealing process, the β vector is the only parameter subject to change. To optimize time, particularly for large problem sizes with extensive J matrices, a separate trigger flag for β is utilized to activate solely the "serialize β " block. Once the parameters are reconstituted in the required format, β is disseminated to a multiplexer module ($N \times 1$ MUX) operating at a 125 MHz clock, which sequentially outputs individual beta values. Simultaneously, J and h are distributed to a module responsible for calculating the field effect around node i (I_i). The fixed-point precision of I_i is adapted based on the maximum number of neighbors, in accordance with the formula depicted in Figure 1. Utilizing a digital signal processing slice (DSP) IP within the FPGA, I_i undergoes scaling with the β value from the MUX. The outcome of this fixed-point multiplication is a change in precision, where the output bit precision becomes the sum of the precisions of the multiplicand and multiplier.

By employing the Metropolis algorithm alongside the Boltzmann distribution, the value of node i is updated through a comparison between $\tanh$ and a value generated by a pseudo-random number generator (PRNG). It is important to note that the bit precision of the scaled I_i is aligned with the precision utilized by the $\tanh$ look-up table. This iterative process continues until all β values have been exhausted, after which the final values of each node in the graph are stored in BRAMs. In the final step, MATLAB retrieves these values to compute the system's energy.

The ensuing discussion will delve into the "new" blocks introduced to a preceding design developed by Navid, specifically, the multiplexer (MUX) (also known as the beta_counter module) and the DSP multiplication block (also known as the beta_scaler module). The focal points of this discussion will be the algorithms implemented within the FPGA and the mechanisms governing their operation.

2.1 Beta Counter Module

```

1: Inputs:
2:   GE: Global Enable flag (Flag that p-bits started).
3:   weight_load_DONE: Weight Load Done flag ( Flag that  $J, h, \beta$  are loaded).
4:   beta_v: Array of beta values.
5:   CLK: Clock signal (125 MHz).
6:   RESET: Reset signal.
7:   num_clk_cycles: Number of clock cycles for each beta value ( # of sweeps/beta).
8:   num_beta: Number of beta values.
9: Output:
10:  beta: Single beta value output.
11: Initialize counter and clk_counter to 0.                                ▷ Reset counters
12: On each positive edge of CLK:
13:   If RESET is high:
14:     Set counter and current_beta_index to 0.                            ▷ Reset counters
15:     beta ← beta_v[0]
16:   Elseif (GE and weight_load_DONE):
17:     If counter == num_clk_cycles - 1:                                    ▷ Check clock cycles
18:       counter ← 0.                                                       ▷ Move to the next beta
19:     If current_beta_index < num_beta - 1
20:       beta ← beta_v[current_beta_index + 1]                             ▷ Assign the next value of beta_v to beta
21:       current_beta_index ← current_beta_index + 1;
22:   Else:
23:     counter ← counter + 1.                                                ▷ Increment counter-same  $\beta$ 

```

Algorithm 1: Beta Counter Module

In this module, we output the beta value on each positive edge of a 125 MHz clock. Each beta value persists for a certain number of sweeps, determined by the number of clock cycles, which is a freely adjustable parameter that we control from MATLAB. Once all sweeps per beta value are exhausted, another beta value is output by incrementing the `counter` parameter. To ensure alignment with the p-bits module and verify that the weights are loaded and available in the BRAM, we condition on two flags: a global enable flag (`GE`), and a weight load completion flag (`weight_load_DONE`). The former indicates that the p-bits can start flipping, while the latter confirms that the weights, including J , h , and β , are available in the BRAM.

2.2 Beta Scaler Module

```

1: Inputs:
2:   unscaled_Iin: Signed fixed-point, word length =  $\mathbf{h\_bit\_width} + \lceil \log_2(N + 1) \rceil$ . ▷ Fixed-point
   precision:  $s[\underbrace{a + \lceil \log_2(N + 1) \rceil}_w][b]$ 
3:   beta: Signed fixed-point, word length =  $\mathbf{beta\_bit\_width}$ . ▷ Fixed-point precision:  $s[c][d]$ 
4: Output:
5:   scaled_Iin: Signed fixed-point, word length =  $\mathbf{i\_bit\_width}$ . ▷ Fixed-point precision:  $s[e][f]$ 
6: procedure BETASCALER(unscaled_Iin, beta)
7:   full_precision_result  $\leftarrow$  unscaled_Iin  $\times$  beta ▷ Fixed-point precision:  $s[w + c + 1][b + d]$ 
8:   scaled_Iin_temp  $\leftarrow$  full_precision_result  $\ggg$  right_shift_bits ▷ Dropping the extra fraction bits.
   Fixed-point precision:  $s[w + c + 1][f]$ .
9:   if scaled_Iin_temp > pmax then
10:     scaled_Iin  $\leftarrow$  maximum value ▷ Clipping to positive maximum of  $s[e][f]$ 
11:   else if scaled_Iin_temp < nmin then
12:     scaled_Iin  $\leftarrow$  minimum value ▷ Clipping to negative minimum of  $s[e][f]$ 
13:   else
14:     scaled_Iin  $\leftarrow$  scaled_Iin_temp[i_bit_width] ▷ Fixed-point precision:  $s[e][f]$ 
15:   end if
16: end procedure

```

Algorithm 2: Beta Scaler Module

In this module, the inverse-temperature p-bit input multiplication ($\beta_i I_i$) is performed using the DSP slice within the FPGA. The multiplication is initially carried out with full precision, which is the sum of the precision of the multiplicand and multiplier. Subsequently, the result is scaled to the fixed-point precision of the Look-up Table (LUT) for tanh, typically $s[4][3]$, i.e., 4 bits for the integer part and 3 bits for the fractional part. This precision is sufficient as tanh saturates at this level, and no additional precision beyond this is necessary. If the full precision result of the multiplication exceeds the precision of the LUT, it is clamped to the maximum value of the LUT's full precision, whether it is the maximum positive value or the minimum negative value. For the integer part, if the value exceeds the LUT's precision, it is clamped to the maximum value supported by the integer bit precision of the LUT. For the fractional part, if there are more bits than the LUT's fractional bit precision, the extra fraction bits are simply discarded by right shift registers, rather than clamping to the largest value supported by the LUT's fractional bit precision.

3 FPGA vs CPU Annealing ($L = 3$)

In this section, we investigate the system energy evolution using MCMC coupled with a simulated annealing algorithm on CPU and FPGA for two schedules, namely, linear, and geometric schedules. The problem size is $L = 3$, that is 27 spins. Please refer to Appendix A for details about the sampling mechanism in CPU and FPGA, and reproducing the same results.

3.1 Linear Schedule

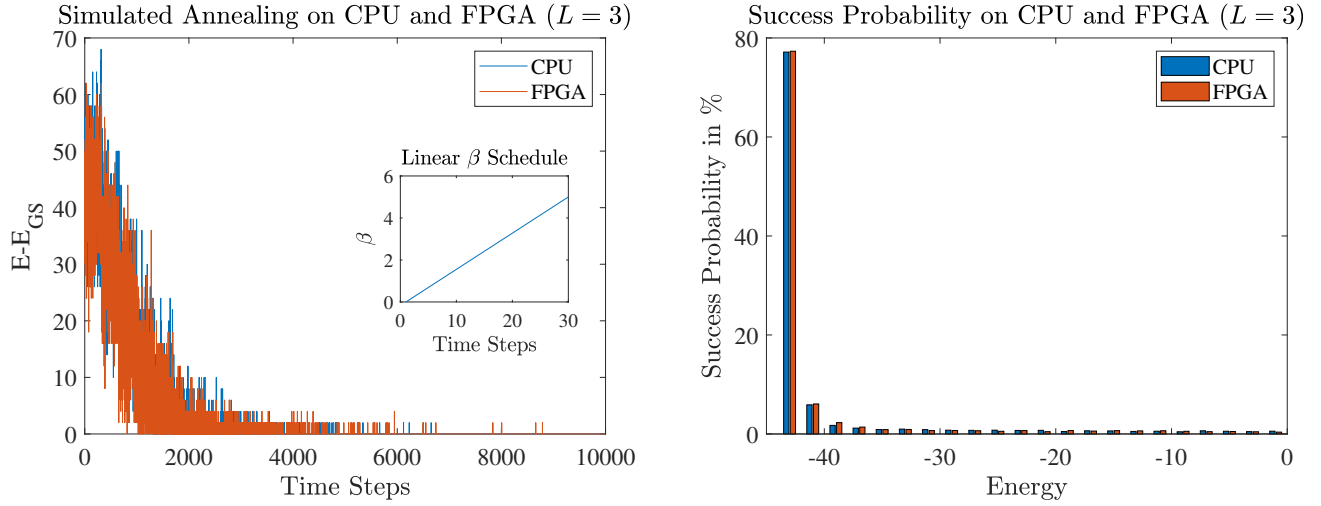


Figure 2: Simulated Annealing on CPU and FPGA using 30 beta values that are linearly spaced. (Left) Excitation energy versus time steps. (Right) Success probability versus energy.

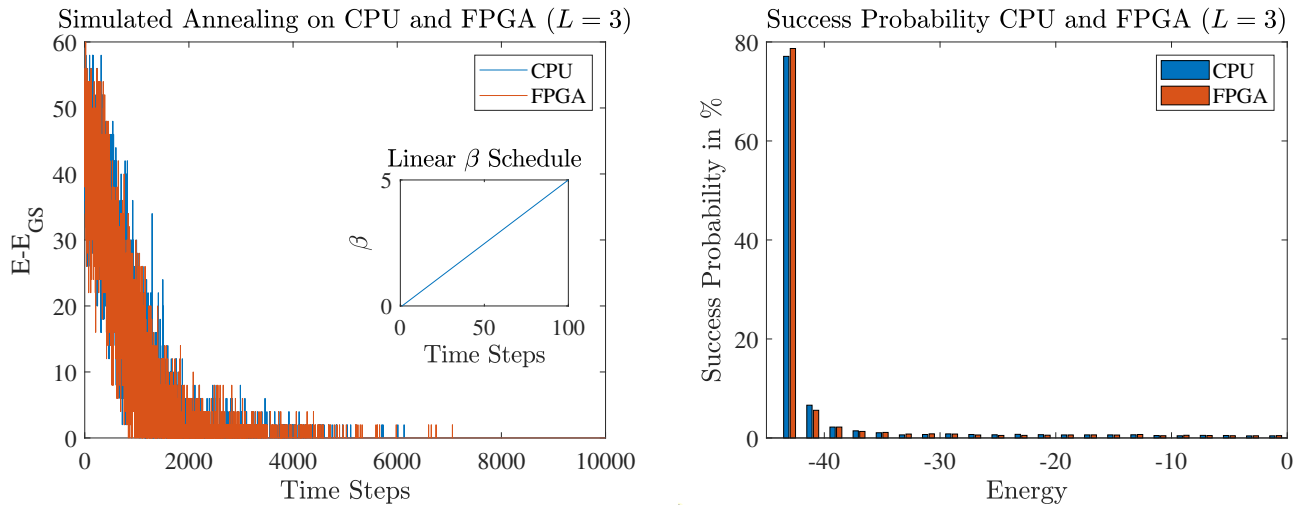


Figure 3: Simulated Annealing on CPU and FPGA using 100 beta values that are linearly spaced. (Left) Excitation energy versus time steps. (Right) Success probability versus energy.

3.2 Geometric Schedule

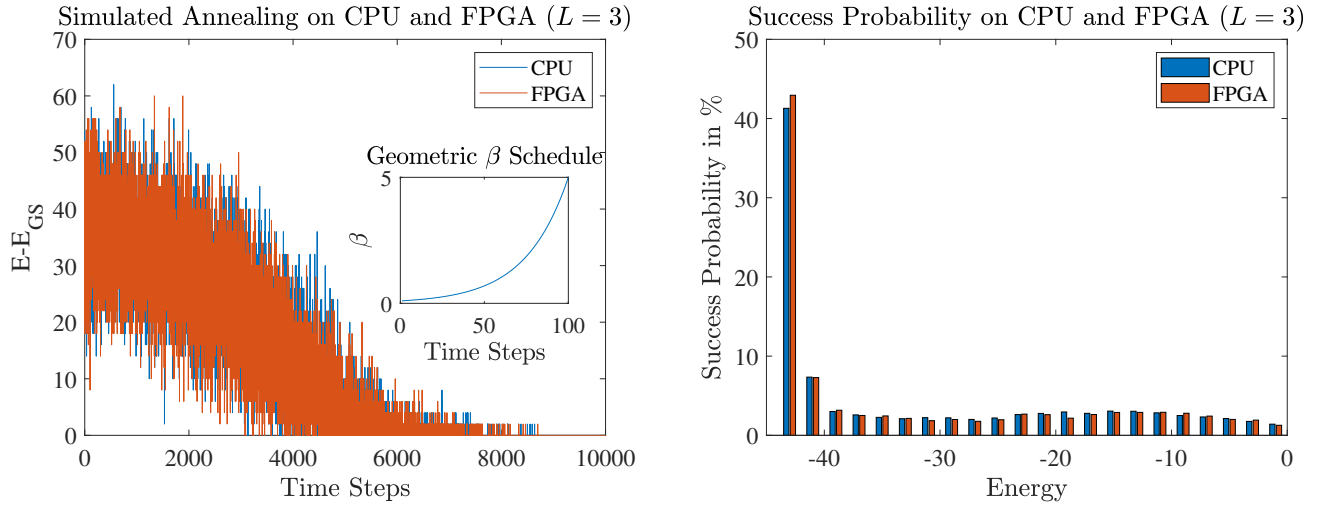


Figure 4: Simulated Annealing on CPU and FPGA using 100 beta values that are geometrically spaced. (Left) Excitation energy versus time steps. (Right) Success probability versus energy.

4 FPGA vs CPU Annealing ($L = 5$)

Similar to the previous section, we further examine the annealing dynamics by sampling the states for a larger problem size ($L = 5$) with 125 spins.

4.1 Linear Schedule

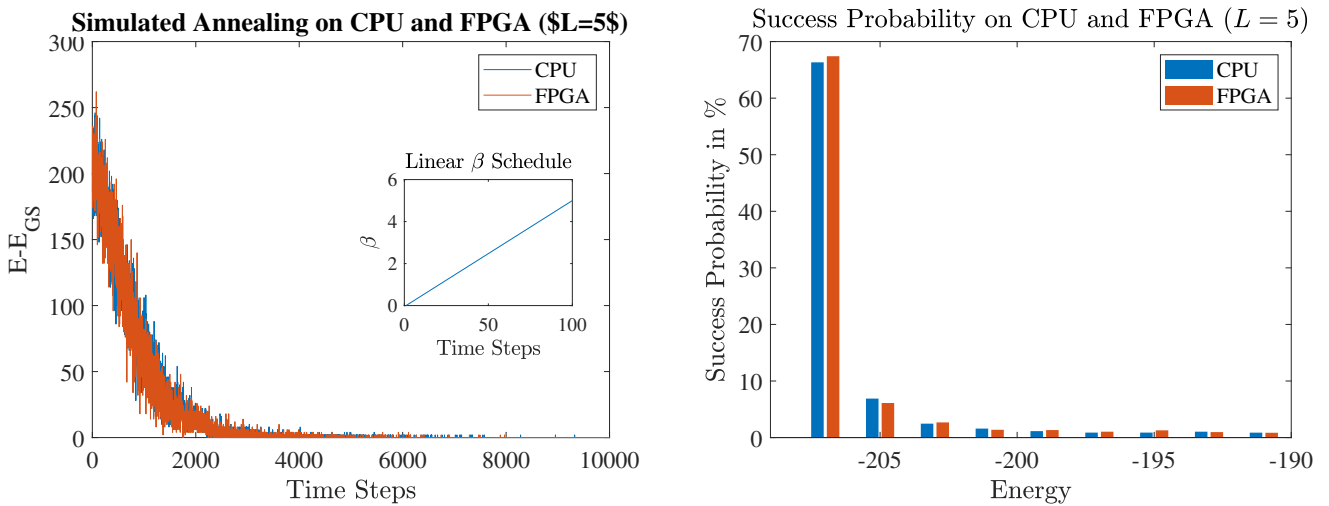


Figure 5: Simulated Annealing on CPU and FPGA using 100 beta values that are linearly spaced. (Left) Excitation energy versus time steps. (Right) Success probability versus energy.

4.2 Geometric Schedule

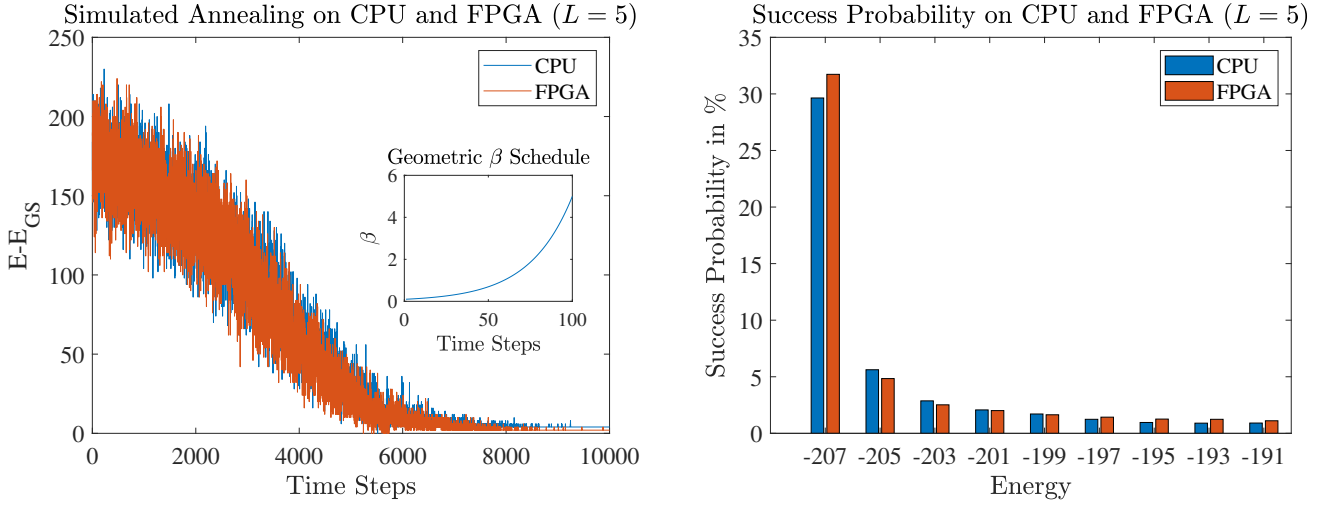


Figure 6: Simulated Annealing on CPU and FPGA using 100 beta values that are geometrically spaced. (Left) Excitation energy versus time steps. (Right) Success probability versus energy.

5 Benchmarking Analysis

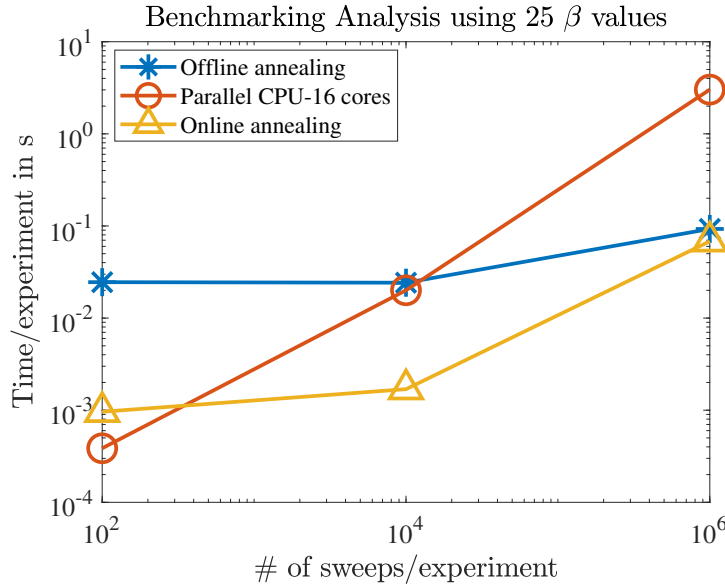


Figure 7: Simulated annealing benchmarking using 25 β values and $L = 3$: Time/experiment averaged over 1000 independent experiments vs. the total # of sweeps/experiment using offline, CPU, and online annealing (the proposed design).

In Figure 7, the average time consumed in a full annealing experiment is plotted against the total number of sweeps for a single experiment, comparing the performance of a CPU with 16 workers, an offline FPGA where the multiplication is performed outside the FPGA, and the proposed online FPGA design. The

CPU’s time consumption exhibits a linear relationship with the number of sweeps. However, the time consumption for both offline and online FPGA designs remains constant from 100 to 10,000 sweeps, indicating that the primary time factor is the fixed delay caused by reading from and writing to the FPGA. Notably, using one million sweeps per experiment does not show any advantage of the online FPGA over the offline FPGA for a β vector with 25 distinct values, suggesting that the time spent loading a single β value at a time from MATLAB is compensated by the annealing time within the FPGA. It is important to note that in the offline FPGA scenario, J and h must be broadcasted for every β value, causing additional delays for large-size problems not present in the online FPGA case. Therefore, the time profile may differ significantly for different problem sizes and could reveal a clear advantage of the online FPGA over the offline FPGA at a substantially large number of sweeps.

6 IP Usability

Given the adjacency matrix of the graph, i.e., J matrix, the following modules need to be generated using the MATLAB codes available under the “**MATLAB-Verilog generator scripts**” page in the GitHub repository:

1. `beta_scaler_module`
2. `pbit_module`
3. `pbitc_module`
4. `weight_module`
5. `LUT_fi`
6. `hardware_layout_fps_BRAM_triu`

These files can be customized according to specific requirements. For instance, the Pseudo-Random Number Generator (PRNG) can be either Linear Feedback Shift Register (LFSR) or Xoshiro. Additionally, one can choose between short and full Look-up Tables (LUT) of $\tanh$. The necessary fixed-point bit precision also needs to be specified in these files. The text files generated for items 1 to 5 will replace the SystemVerilog files with the same names plus “.sv” available under the “**Online Annealing Verilog Project**” page in the GitHub repository. However, the file generated by item 6 will only replace the instances of `pbit` and `pbitc` modules lines in the top module called `PSL`. Finally, the number of beta values, pbits, the length of the J matrix, and available clocks (unique colors) are generated by `hardware_layout_fps_BRAM_triu` and need to be included in the `PSL_pkg.sv` module.

7 Conclusion

In conclusion, this report has presented an innovative design for fast simulated annealing on FPGA hardware, emphasizing the strategic postponement of communication between the FPGA and MATLAB to the final phase of the annealing process. The proposed system's architecture and core algorithms have been thoroughly outlined, providing a clear understanding of its operational framework. The system's dynamics have been validated using energy plots and success probability metrics, with experiments conducted on two distinct instances of 3D-Spin Glass problems of varying sizes. The results have demonstrated the efficiency of the proposed design, particularly in terms of time consumption, where the online FPGA design showed a constant time profile as opposed to the linear relationship exhibited by the CPU.

Furthermore, the benchmarking analysis has highlighted the potential advantages of the online FPGA design over the offline FPGA and CPU, especially at significantly large numbers of sweeps. The IP usability section has offered a step-by-step guide for utilizing the proposed IP for general problem settings, facilitating its adoption for various applications and by new users.

The report has also delved into specific modules required for the system's operation, detailing the generation of these modules using MATLAB codes and their customization options. The process of replacing system Verilog files with the generated text files has been explained, ensuring a seamless integration of the new design into the existing framework.

8 Recommendations

A potential enhancement to the current algorithm could be realized by allowing each node to possess a distinct inverse-temperature parameter, β . In the present implementation, a universal β value is employed to scale the calculated field effects of all nodes uniformly. However, by assigning a unique β value to each field effect, I_i , we introduce additional degrees of freedom that could potentially expedite the convergence to the ground energy state. The implementation of this concept requires further investigation and collaborative efforts to explore its feasibility and effectiveness.

A Simulated Annealing Sampling Algorithms (CPU and FPGA)

- 1: Define clocks: `ref_clk = 125 MHz`, `pbit_clk = 15 MHz`, `bram_read_clk = 100 MHz`
- 2: Define memory interface: `mem`
- 3: Define addresses for FPGA registers
- 4: Load problem specifications: `J`, `h`
- 5: Convert weights to binary and reshape: `J_binary_resaped`, `h_binary`
- 6: Set β schedule
- 7: Define annealing parameters: `NT = 104` (time steps)
- 8: `num_sweeps_per_time_step = 10` ▷ # sweeps/single time step
- 9: `tictoc_counter_limit ← ⌈num_sweeps_per_time_step/pbit_clk × ref_clk⌉` ▷ tictoc_counter module is running at `ref_clk`. Division is needed since # of sweeps is clocked by `pbit_clk`.
- 10: `total_annealing_time ← NT × num_sweeps_per_time_step` ▷ Total # of sweeps for all beta
- 11: `num_sweeps_beta ← total_annealing_time / length(B)`
- 12: `num_clk_limit ← ⌈num_sweeps_beta/pbit_clk × ref_clk⌉` ▷ beta_counter module is running at `ref_clk`. Division is needed since # of sweeps is clocked by `pbit_clk`.
- 13: Write annealing parameters to FPGA
- 14: Convert weights to fixed-point precision: `J_fpga`, `h_fpga`, `beta_fpga`
- 15: Load weights to FPGA
- 16: Reset beta counter
- 17: **for** `step` in `1:NT` **do**
- 18: Reset p-bit counter
- 19: Wait for p-bits to freeze
- 20: Read p-bit states
- 21: Calculate cost
- 22: **end for**

Algorithm 3: FPGA-Based Simulated Annealing

- 1: Load problem specifications: J, h
- 2: Define β schedule
- 3: Repeat each beta value 100 times to form B ▷ To get NT samples as in FPGA
- 4: Initialize random state m
- 5: **for** $step$ in $1:\text{length}(B)$ **do**
- 6: **for** i in $1:\text{size}(J,1)$ **do**
- 7: Calculate input I for spin i
- 8: Update spin $m(i)$ using I
- 9: **end for**
- 10: Calculate energy $E(step)$
- 11: **end for**
- 12: Calculate unique energies and success probabilities for CPU and FPGA

Algorithm 4: Simulated Annealing on CPU